Package 'skiLift'

October 29, 2026

Title DBI Interface to 'Snowflake'

Version 0.2.1

Description A DBI-compliant interface to Snowflake (<https://www.snowflake.com/en/>). Connects via Snowflake's SQL API v2 (HTTPS/JSON) as the universal baseline, with no dependency on ODBC, JDBC, or Python. Supports key-pair JWT, programmatic access token (PAT), OAuth, and session token authentication. Optionally uses ADBC (Arrow Database Connectivity) as an accelerator for bulk reads (Arrow-native) and writes (PUT + COPY INTO), activated automatically when adbcsnowflake is installed.

License Apache License 2.0 | file LICENSE

URL https://github.com/posit-dev/skiLift

BugReports https://github.com/posit-dev/skiLift/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1.0)

Imports DBI (>= 1.2.0),
methods,
httr2 (>= 1.0.0),
jsonlite,
rlang (>= 1.0.0),
cli,
openssl

Suggests bit64,
jose,
RcppTOML,
RcppSimdJson,
nanoarrow (>= 0.3.0),
adbcdrivermanager,
adbcsnowflake,
dbplyr (>= 2.3.0),
dplyr,
duckdb,
blob (>= 1.2.0),
hms (>= 1.0.0),
reticulate,

snowflakeauth,
httpuv,
DBItest (>= 1.7.0),
testthat (>= 3.0.0),
mockr,
withr,
knitr,
rmarkdown

VignetteBuilder knitr

Config/testthat/edition 3

Contents

rcat . . . 2
rglimpse . . . 3
rprint . . . 3
rview . . . 4
sf_cache_in_duckdb . . . 4
sf_collect_to_duckdb . . . 5
sf_connect . . . 5
sf_duckdb_tables . . . 6
skilift-errors . . . 6
skiLift-options . . . 8
Snowflake . . . 8
SnowflakeConnection-class . . . 9
SnowflakeDriver-class . . . 13
SnowflakeResult-class . . . 15

`rcat` *Clean replacement for cat() in Workspace Notebooks*

Description

Builds the full string first, then outputs with `writeLines()` to avoid the extra line breaks that Workspace Notebooks add.

Usage

```
rcat(..., sep = "")
```

Arguments

`...`	Objects to concatenate and print.
`sep`	Separator between arguments. Default: "".

Value

Invisibly returns `NULL`.

rglimpse *Glimpse a data.frame structure*

Description

Glimpse a data.frame structure

Usage

```
rglimpse(x)
```

Arguments

x	A data.frame.

Value

Invisibly returns `x`.

rprint *Print a data.frame cleanly in Workspace Notebooks*

Description

Captures all output lines and writes them as a single block, avoiding the extra whitespace that Workspace Notebooks insert between lines.

Usage

```
rprint(x, width = 200L, ...)
```

Arguments

x	A data.frame or other printable object.
width	Integer. Print width in characters. Default: 200.
...	Additional arguments passed to `print()`.

Value

Invisibly returns `x`.

rview *View the head of a data.frame cleanly*

Description

View the head of a data.frame cleanly

Usage

```
rview(x, n = 10L, width = 200L)
```

Arguments

x	A data.frame.
n	Integer. Number of rows to show. Default: 10.
width	Integer. Print width in characters. Default: 200.

Value

Invisibly returns the first n rows.

sf_cache_in_duckdb *Cache a Snowflake query result in a local DuckDB table*

Description

Executes sql against Snowflake (using the best available backend – ADBC if installed, REST API v2 otherwise) and writes the result into a DuckDB table for fast local analysis with dplyr / dbplyr.

Usage

```
sf_cache_in_duckdb(sf_con, sql, table_name, duck_con = NULL, overwrite = TRUE)
```

Arguments

sf_con	A SnowflakeConnection object.
sql	Character SQL query to run against Snowflake.
table_name	Character name for the DuckDB table.
duck_con	An existing DuckDB connection, or NULL to create one.
overwrite	Logical; overwrite if the table already exists? Default TRUE.

Value

The DuckDB connection (invisibly). If duck_con was NULL, the caller is responsible for disconnecting it.

sf_collect_to_duckdb *Materialise a* dplyr *lazy table from Snowflake into DuckDB*

Description

Takes a tbl backed by a Snowflake connection, collects the underlying SQL, executes it against Snowflake, and writes the result into DuckDB for continued dplyr analysis.

Usage

```
sf_collect_to_duckdb(.tbl, table_name, duck_con = NULL, overwrite = TRUE)
```

Arguments

.tbl	A tbl_lazy backed by a SnowflakeConnection.
table_name	Character name for the DuckDB table.
duck_con	An existing DuckDB connection, or NULL to create one.
overwrite	Logical; overwrite if the table already exists? Default TRUE.

Value

A tbl backed by DuckDB, ready for further dplyr ops.

sf_connect *Connect to Snowflake using a profile name*

Description

Convenience wrapper around dbConnect(Snowflake(), name = ...).

Usage

```
sf_connect(name = NULL, ...)
```

Arguments

name	Profile name from connections.toml.
...	Additional arguments passed to dbConnect.

Value

A SnowflakeConnection.

sf_duckdb_tables *List tables cached in a DuckDB connection*

Description

List tables cached in a DuckDB connection

Usage

```
sf_duckdb_tables(duck_con)
```

Arguments

duck_con	A DuckDB connection.

Value

Character vector of table names.

skilift-errors *Errors raised by skiLift*

Description

Errors that come from Snowflake, or from failing to reach it, have a class and carry the facts as data, so code can tell an authentication failure from a SQL error from a timeout without reading the message.

Classes

Every error has class `skilift_error`. More specific classes come first.

- `skilift_error_config`: a problem found locally, before any request (bad arguments, a row too large to send, transactions, which skiLift does not support).
- `skilift_error_http`: Snowflake answered with an error status. Also one of:
  - `skilift_error_auth`: 401, after the credentials were refreshed once.
  - `skilift_error_forbidden`: 403.
  - `skilift_error_sql`: 422, the SQL did not compile or run.
- `skilift_error_timeout`: a statement did not finish within `options(skiLift.timeout)` (it is then cancelled), or a request timed out.
- `skilift_error_retry`: a temporary failure (429, 500, 502, 503, 504, or no response at all) lasted through every retry. The first cause is in `parent`.
- `skilift_error_write`: a write with `dbWriteTable()` or `dbAppendTable()` stopped partway.

Fields

These are stored on the condition, and are `NULL` when not known.

- `status`: the HTTP status.
- `code`: Snowflake's error code, for example `"002003"`.
- `sql_state`: the SQL state, for example `"42S02"`.
- `statement_handle`: the statement's id, which is its query id in Snowflake's Query History.
- `request_id`: the id skiLift sent with the request. Quote it when asking Snowflake support about a statement.
- `attempts`: how many times the request was sent.

An error of class `skilift_error_write` also has `outcome`, `rows_committed`, `batches_committed`, `total_rows` and `failed_batch` (`index`, `first_row`, `n_rows`). `outcome` says what is true of the failed batch:

- `"failed"`: Snowflake ran it and it failed, so it is not in the table.
- `"not_executed"`: it was rejected before it ran, so it is not in the table.
- `"unknown"`: the retries ran out or the request timed out, so it **may** have been committed. Look up `request_id` in Snowflake's Query History before writing the rows again, or they may be duplicated.

Retries

A statement is sent with a `requestId`. If the request has to be sent again, the same id is sent with `retry=true`, and Snowflake does not run the statement a second time if it already ran. So a retried `INSERT` does not write its rows twice. Only temporary failures are retried; a SQL error, 400, 401 (after one credential refresh) and 403 are not. See skiLift-options for the limits.

Examples

```
## Not run:
result <- tryCatch(
  dbGetQuery(con, "SELECT * FROM no_such_table"),
  skilift_error_sql = function(e) {
    message("Snowflake says ", e$code, " (SQL state ", e$sql_state, ")")
    NULL
  },
  skilift_error_timeout = function(e) {
    message("Look up ", e$statement_handle, " in Query History")
    NULL
  }
)

## End(Not run)
```

skiLift-options *skiLift options for timeouts, retries and batch writes*

Description

Set with `options()`. Each is read when it is used, so changing one takes effect for the next statement.

Options

- `skiLift.timeout` (default `600`): seconds a statement may take, measured from when it is submitted. It is passed to Snowflake as the statement timeout, and it also bounds skiLift's own waiting and retrying. A statement still running at the deadline is cancelled and raises an error of class `skilift_error_timeout`. This is one limit for the whole statement, not one per attempt.
- `skiLift.request_timeout` (default `90`): seconds one request may wait for Snowflake to answer before it is abandoned and sent again. Snowflake answers a statement within about 45 seconds (with the result, or with a handle if it is still running) and a status check within seconds, so a request silent for 90 seconds has hung. Without this limit a hung request waited out all of `skiLift.timeout`. The resend carries the same request id, so it does not run the statement twice. Downloading a large result partition is not limited by it.
- `skiLift.retry_max` (default `3`): how many times a request is sent, in all, when it fails in a way that is safe to retry. The wait between attempts grows (about 0.5, 1, 2, ... seconds, with some randomness) and is at least the `Retry-After` Snowflake asks for.
- `skiLift.insert_batch_size` (default `16384`): the most rows in one `INSERT` statement when writing a data frame.
- `skiLift.insert_batch_bytes` (default `1e6`): the most bytes of SQL in one `INSERT`. A batch ends at whichever of the two limits comes first. Snowflake advises keeping a request near 1 MB; wide rows or long text reach this limit long before the row limit. A single row larger than this cannot be sent and is an error.

Waiting for long statements

Snowflake answers a statement that runs for more than 45 seconds with a handle instead of a result. skiLift then checks again, waiting longer each time (from 0.5 up to 5 seconds), until the statement has finished, so a slow query or a slow `INSERT` returns only when it is done. Pressing Esc while waiting cancels the statement in Snowflake so it does not keep running.

Snowflake *Create a SnowflakeDriver instance*

Description

Create a SnowflakeDriver instance

Usage

```
Snowflake()
```

Value

A SnowflakeDriver singleton.

Examples

```
## Not run:
drv <- Snowflake()
con <- dbConnect(drv, account = "myaccount", name = "default")

## End(Not run)
```

SnowflakeConnection-class

SnowflakeConnection

Description

An S4 class representing a connection to Snowflake via the SQL API v2. Created by `DBI::dbConnect()` with a `Snowflake()` driver.

Usage

```
## S4 method for signature 'SnowflakeConnection'
show(object)

## S4 method for signature 'SnowflakeConnection'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeConnection'
dbDisconnect(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendQuery(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendStatement(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbGetQuery(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbExecute(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbGetQueryArrow(conn, statement, params = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbSendQueryArrow(conn, statement, params = NULL, ...)
```

```
## S4 method for signature 'SnowflakeConnection'
dbListTables(conn, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbExistsTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbListFields(conn, name, ...)

## S4 method for signature 'SnowflakeConnection'
dbListObjects(conn, prefix = NULL, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbReadTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character,data.frame'
dbWriteTable(
  conn,
  name,
  value,
  overwrite = FALSE,
  append = FALSE,
  row.names = FALSE,
  field.types = NULL,
  ...
)

## S4 method for signature 'SnowflakeConnection,character'
dbCreateTable(conn, name, fields, ..., row.names = NULL, temporary = FALSE)

## S4 method for signature 'SnowflakeConnection,Id'
dbAppendTable(conn, name, value, ..., row.names = NULL)

## S4 method for signature 'SnowflakeConnection,Id'
dbCreateTable(conn, name, fields, ..., row.names = NULL, temporary = FALSE)

## S4 method for signature 'SnowflakeConnection,character'
dbAppendTable(conn, name, value, ..., row.names = NULL)

## S4 method for signature 'SnowflakeConnection,character'
dbRemoveTable(conn, name, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbQuoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,character'
dbQuoteString(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,SQL'
dbQuoteString(conn, x, ...)
```

```
## S4 method for signature 'SnowflakeConnection,SQL'
dbQuoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection,SQL'
dbUnquoteIdentifier(conn, x, ...)

## S4 method for signature 'SnowflakeConnection'
dbQuoteLiteral(conn, x, ...)

## S4 method for signature 'SnowflakeConnection'
dbDataType(dbObj, obj, ...)

## S4 method for signature 'SnowflakeConnection'
sqlData(con, value, row.names = FALSE, ...)

## S4 method for signature 'SnowflakeConnection'
dbBegin(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbCommit(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbRollback(conn, ...)

## S4 method for signature 'SnowflakeConnection'
dbWithTransaction(conn, code, ...)
```

Arguments

object	A SnowflakeConnection object (for show).
dbObj	A SnowflakeConnection object.
...	Additional arguments (currently ignored).
conn	A SnowflakeConnection object.
statement	SQL statement string.
params	Optional parameter list for parameterized queries.
name	Table name (character).
prefix	An Id object indicating the hierarchy level to list (NULL for databases, 1-component for schemas, 2-component for tables).
value	A data.frame.
overwrite	Logical. Drop existing table first?
append	Logical. Append to existing table?
row.names	Not supported: row names are never written. TRUE or a column name gives a warning, and FALSE, NA and NULL are accepted.
field.types	Named character vector of Snowflake column types that replace the types skiLift would choose, for example c(id = "BIGINT", note = "VARCHAR(50)"). Names must be columns of value.
fields	A data.frame or named character vector of column types.
temporary	Logical. Create a temporary table?

x	Character to quote.
obj	An R object to map to a Snowflake SQL type.
con	A SnowflakeConnection object.
code	Code to execute within the transaction.

Slots

account Account identifier.

user Username.

database Default database.

schema Default schema.

warehouse Default warehouse.

role Default role.

.auth Auth list (type, token, token_type, plus params for refresh).

.state Environment holding mutable state (valid, in_transaction, session_info, and the live auth token cache – token, token_mtime).

Writing data frames

dbWriteTable() and dbAppendTable() send rows as batched INSERT statements. A batch holds at most options(skiLift.insert_batch_size) rows and options(skiLift.insert_batch_bytes) bytes of SQL, whichever is reached first (see skiLift-options).

A write is not atomic. Each batch commits on its own, so if batch 7 of 10 fails, batches 1 to 6 stay in the table (and dbWriteTable() has already created the table). The error, of class skilift_error_write, says how many rows were committed, which rows were in the failed batch, and whether that batch may have been committed too (outcome); see skilift-errors. A batch that is sent again after a temporary failure carries the same request id, so Snowflake does not run it twice.

Transactions

skiLift does not support transactions. dbBegin(), dbCommit(), dbRollback() and dbWithTransaction() raise an error of class skilift_error_config. Each statement is sent to the Snowflake SQL API on its own and committed as it runs, so there is nothing for a BEGIN to hold open. To make changes that must all succeed or all fail, send them as one Snowflake Scripting block:

```
dbExecute(con, "EXECUTE IMMEDIATE $$
  BEGIN
    BEGIN TRANSACTION;
    INSERT INTO t VALUES (1);
    INSERT INTO t VALUES (2);
    COMMIT;
  END
$$")
```

If a statement in the block fails, the block fails and nothing is committed. Writes with dbWriteTable() and dbAppendTable() are sent in batches, each committed on its own, so they are not atomic either; see ?dbWriteTable.

SnowflakeDriver-class *SnowflakeDriver*

Description

An S4 class representing the Snowflake DBI driver.

Usage

```
## S4 method for signature 'SnowflakeDriver'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeDriver'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeDriver'
dbUnloadDriver(drv, ...)

## S4 method for signature 'SnowflakeDriver'
show(object)

## S4 method for signature 'SnowflakeDriver'
dbDataType(dbObj, obj, ...)

## S4 method for signature 'SnowflakeDriver'
dbConnect(
  drv,
  account = NULL,
  user = NULL,
  token = NULL,
  private_key_path = NULL,
  authenticator = NULL,
  database = "",
  schema = "",
  warehouse = "",
  role = "",
  name = NULL,
  bigint = getOption("skiLift.bigint", "numeric"),
  timezone_out = getOption("skiLift.timezone_out", "database"),
  ...
)

## S4 method for signature 'SnowflakeDriver'
dbCanConnect(drv, ...)
```

Arguments

dbObj	A SnowflakeDriver object.
...	Additional arguments (ignored).
drv	A SnowflakeDriver, or missing (uses default).

`object` A SnowflakeDriver object (for show).

`obj` An R object to map to a Snowflake SQL type.

`account` Snowflake account identifier (e.g. "myaccount").

`user` Snowflake username.

`token` Explicit bearer token (PAT or session token).

`private_key_path` Path to PEM private key for JWT auth.

`authenticator` Auth method ("SNOWFLAKE_JWT", "OAUTH", etc.).

`database` Default database.

`schema` Default schema.

`warehouse` Default warehouse.

`role` Default role.

`name` Profile name from connections.toml.

`bigint` How to return whole-number columns too large for an R integer. Snowflake stores `INTEGER`, `BIGINT` and `NUMBER(38,0)` the same way, so a column's type is decided by its precision, never by the values in a particular result. Columns of up to 9 digits are always `integer`. Larger ones, including every plain `INTEGER` column, follow `bigint`: `"numeric"` (the default; exact up to 2^53, which covers any ordinary ID or count), `"integer64"`, `"integer"` or `"character"`. `"integer64"` (needs the `bit64` package) is exact for any 64-bit value, but many R functions do not understand it and some give wrong answers without a warning – `lm()`, for example – so choose it only if you need values above 2^53. A value that does not fit the chosen type is returned as `NA` with a warning. To change the default for every connection, set `options(skiLift.bigint = "integer64")`.

`timezone_out` The time zone `TIMESTAMP_LTZ` and `TIMESTAMP_TZ` values are shown in. `"database"` (the default) is the Snowflake session's `TIMEZONE` setting, the same zone Snowflake's own SQL functions and its Python and ADBC drivers use, so an hour-of-day computed in R agrees with one computed in a Snowflake query. `"local"` is your R session's time zone, and any name from `OlsonNames()` (such as `"UTC"`) is used as given. It only changes how a time is labelled, never which moment it is. `TIMESTAMP_NTZ` values have no zone and are labelled `"UTC"`. To change the default for every connection, set `options(skiLift.timezone_out = "UTC")`.

Connecting with no arguments

`dbConnect(Snowflake())` can be called with **no arguments at all** inside the Posit Team Native App (Native App) or Posit Workbench. Both write their own `connections.toml` profile before your code runs; when `account` is `NULL`, `dbConnect()` reads that profile automatically. The connection authenticates as the Snowflake user who launched the Native App session – not a shared service account – in that user's default role, warehouse, and database, unless you override any of `account`, `user`, `warehouse`, `database`, `schema`, or `role` explicitly. An explicit argument always wins over the profile; the profile always wins over an empty default.

SnowflakeResult-class *SnowflakeResult*

Description

An S4 class representing the result of a query against Snowflake. Created by DBI::dbSendQuery() or DBI::dbSendStatement().

Usage

```
## S4 method for signature 'SnowflakeResult'
show(object)

## S4 method for signature 'SnowflakeResult'
dbIsValid(dbObj, ...)

## S4 method for signature 'SnowflakeResult'
dbClearResult(res, ...)

## S4 method for signature 'SnowflakeResult'
dbFetch(res, n = -1, ...)

## S4 method for signature 'SnowflakeResult'
dbHasCompleted(res, ...)

## S4 method for signature 'SnowflakeResult'
dbFetchArrow(res, ...)

## S4 method for signature 'SnowflakeResult'
dbFetchArrowChunk(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetRowCount(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetRowsAffected(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetStatement(res, ...)

## S4 method for signature 'SnowflakeResult'
dbColumnInfo(res, ...)

## S4 method for signature 'SnowflakeResult'
dbGetInfo(dbObj, ...)

## S4 method for signature 'SnowflakeResult'
dbBind(res, params, ...)
```

Arguments

object	A SnowflakeResult object (for show).

`dbObj`	A SnowflakeResult object.
`...`	Additional arguments (currently ignored).
`res`	A SnowflakeResult object.
`n`	Number of rows to fetch. -1 (default) fetches all.
`params`	A list of parameter values for parameterized queries.

Slots

`connection` The parent SnowflakeConnection.

`statement` The SQL statement string.

`.resp_body` The raw JSON response from the first partition.

`.meta` Parsed metadata (columns, partitions, handle).

`.state` Environment holding mutable state (fetched, partition, valid, etc.).